🚀 MedEd-AI → Healthcare AI Engineer — Complete 10/10 Roadmap (clean, integrated)

Overview: this roadmap keeps every phase from your original stack and **adds** the three modules you requested:

- **Module A — AI Product Engineer (Business / Metrics / Rapid Prototyping)** (integrated into Phase 6)
- **Module B — AI Bodyguard (Security / Observability / Liability)** (integrated into Phase 9 & Phase 10)
- **Module C — Deep Anchor Specializations** (added as Phase 11 with three track options)

Follow phases sequentially where marked, but expect overlap: product + security + specialization work should run in parallel with projects.

🟩 PHASE 0 — Engineering Foundations (Non-Negotiable)

Goal: become a reliable software engineer.

0.1 Python Mastery

- fundamentals → advanced OOP, iterators, generators, type hints, dunder methods. 0.2 Environment & Tooling
- VS Code/PyCharm, venv/pip, pip-tools, Git (branching, rebase, PR etiquette). 0.3 Testing & Code Quality
- pytest, unit tests, TDD basics, black/ruff, pre-commit hooks.

Deliverable: small library + tests (published on GitHub), CI that runs tests on PR.

🟩 PHASE 1 — Data + Async Backend Core

Goal: build high-performance, data-driven backends.

1.1 Data Science Core

- NumPy, Pandas, EDA, medical dataset handling (missingness, noise). 1.2 Async Python (critical)

- asyncio, async/await, aiohttp, concurrency patterns.

Deliverable: a small ETL + async worker that ingests a medical CSV, normalizes, and writes to DB.

▧ PHASE 2 — Backend API Engineering (FastAPI)

Goal: production-ready APIs.

2.1 Core stack: FastAPI, Uvicorn, Pydantic models. 2.2 Clean architecture: routers / services / repositories, DI, middleware. 2.3 Background processing: Celery / RQ / FastAPI BackgroundTasks for async AI calls.

Deliverable: documented FastAPI service with modular structure and example endpoints.

▧ PHASE 3 — Data Acquisition & Persistence

Goal: reliable ingestion + storage.

3.1 Web scraping: requests/BeautifulSoup; Playwright for dynamic sites. 3.2 DB engineering: SQLAlchemy, relational modeling, Alembic migrations, indexes. 3.3 Caching & performance: Redis, cache invalidation strategies.

Deliverable: robust ingestion pipeline + reproducible migration + caching layer.

▧ PHASE 4 — DevOps & Deployment (MUST before AI specialization)

Goal: ship and operate software.

4.1 Containerization: Docker multi-stage builds, env separation. 4.2 CI/CD: GitHub Actions pipeline — test, lint, build, push image. 4.3 Cloud basics: deploy to Cloud Run / EC2 / App Service; secrets management.

Deliverable: production deployment (staging + prod), infra as code (basic Terraform or Cloud Run config).

▧ PHASE 5 — AI Integration (Applied LLM Engineering)

Goal: use LLMs reliably and safely.

5.1 LLM APIs: OpenAI (or alternative), function calling, structured JSON outputs. 5.2 Prompt engineering (engineering-grade): schema enforcement + deterministic prompts + fallback logic. 5.3 Backend integration: async LLM calls, retries, rate limits, batching.

Deliverable: a microservice endpoint that takes input, calls an LLM with function calling, validates and stores structured outputs.

▧ PHASE 6 — User Interfaces & Monetization **+ MODULE A (AI Product Engineer)**

Goal: real users, monetization, business judgement.

6.1 UI rapid prototyping (Vibe coding)

- Learn v0.dev / Bolt.new / Lovable or equivalent no-code / low-code UI builders.
- Build clickable demos in <2 hours to validate ideas.

6.2 UX & primary product flows

- Telegram Bot (primary UX) + lightweight web dashboard (Tailwind / comfy templates). 6.3 Payments & monetization
- Stripe / Razorpay integration, webhooks, subscription logic. 6.4 Metrics over Models (MODULE A core)
- Instrument product with PostHog or Mixpanel. Track retention, task completion, DAU/WAU, feature funnels.
- A/B testing: deploy variants (prompt variants, UX variants), collect metrics, measure business impact. 6.5 Domain-Driven Design (DDD) for healthcare
- Learn to model constructs (Patient Visit, Order, Encounter, LabResult) as events/documents/transactions. Create canonical data contracts and schema.

Deliverable: an MVP (Telegram + dashboard) instrumented with PostHog + one A/B test that measures an actual behavior metric (e.g., “clinician accepted recommendation” rate).

▧ PHASE 7 — Machine Learning Foundations

Goal: understand models beyond the API.

7.1 Classical ML: scikit-learn, evaluation metrics (precision/recall/F1), bias/variance. 7.2 Deep Learning: PyTorch essentials, custom training loops. 7.3 Biomedical modeling basics: BioBERT, medical NER.

Deliverable: small model training pipeline (fine-tune a biomedical model) with reproducible results and metrics logged.

▧ PHASE 8 — NLP & Biomedical AI

Goal: domain-specific intelligence.

8.1 Hugging Face Transformers, tokenization, fine-tuning. 8.2 Biomedical models: BioBERT / clinical BERT, NER for PHI, symptom classification. 8.3 Efficient fine-tuning: LoRA/PEFT, cost-aware updates.

Deliverable: a fine-tuned NER or classification model with documented evaluation on a held-out set.

▧ PHASE 9 — MLOps (NEW + MODULE B observability/security)

Goal: make ML maintainable, observable, and cost-controlled.

9.1 Experiment tracking: MLflow / Weights & Biases, reproducible experiments. 9.2 Model registry & versioning: MLflow/MLRun, rollbacks, canary releases. 9.3 Monitoring & drift detection: data drift, prediction drift, label drift detection, automated alerts. **MODULE B — LLM Security & Observability (integrated here)**

- LLM Red Teaming: learn prompt injection patterns, test for jailbreaks. Implement guardrails (NeMo Guardrails / Lakera / custom validators).
- Observability stacks: OpenTelemetry + Grafana/Prometheus + a model-observability tool (LangSmith / Arize) to trace inputs → retriever → model → output.
- Tracing & provenance: instrument each step so you can answer “which step failed?” (retrieval, augment, model, post-process).

- Cost monitoring: implement per-feature cost dashboards; set budget alerts and implement caching/quantization to reduce costs.

Deliverable: production-grade model deployment with tracing, cost dashboards, drift alerts, and documented guardrail tests (red-team report).

▧ PHASE 10 — Healthcare Compliance, Ethics & Liability Management (MODULE B continued)

Goal: build safe systems that pass legal/compliance review.

10.1 Data privacy & governance: HIPAA basics, PHI handling, secure storage, least privilege. 10.2 De-identification & logging hygiene: PII removal, differential privacy basics, secure audit logs. 10.3 Interoperability & auditability: FHIR resources, API audit trails, explainability artifacts for models. 10.4 Liability management (MODULE B): threat modeling for patient harm scenarios, runbooks for incidents, SLAs and error budgets, legal checklists for deployment in clinics.

Deliverable: compliance checklist, one mock security & compliance review (paperwork + runbook), and a signed runbook for an incident scenario.

▧ PHASE 11 — Deep Anchor Specializations (MODULE C)

Goal: pick one anchor to go *very deep* — here we add **all three tracks** (you said include all three). Each should be taken to mastery level sequentially or in parallel only if you have capacity; but the roadmap shows how to add all three so you can be market-flexible.

> NOTE: Depth matters. For career-fortress effect, prioritize **one** anchor to reach high competence, then add the others later.

Track A — Clinical AI Specialist (Hospitals / MedTech)

- FHIR expertise: full parsing + profile mapping + FHIR workflows.
- DICOM & medical imaging pipelines: PACS access, DICOM tags, safe image handling.
- Federated learning & privacy-preserving ML: PySyft / Flower / secure aggregation, governance for hospital data.

- Clinical validation: understand clinical trial basics, sensitivity/specificity tradeoffs, clinical endpoints. Deliverable: integrated POC that consumes FHIR resources and outputs clinically validated alerts; DICOM processing pipeline with audit trail.

Track B — LLM Systems Architect (Thinking systems)

- RAG advances: GraphRAG, HyDE (hypothetical document embeddings), reranking strategies.
- Agent orchestration: LangGraph / custom orchestrators for stateful agents with retries and memory.
- Inference optimization: vLLM, quantization (bitsandbytes), batching, sharded inference.
- System design: latency vs. cost tradeoffs, fallback strategies, progressive disclosure UI. Deliverable: a production proof of concept of a stateful agent orchestration with RAG + reranker + cost-aware inference serving.

Track C — MLOps Engineer (Infrastructure & Reliability)

- Kubernetes for ML: KServe, Ray Serve, KFServing patterns.
- Feature stores: Feast, real-time feature pipelines, schema evolution.
- Model registry & CI/CD: MLflow + GitOps for model promotion.
- Chaos & reliability: load testing, rollback automation, runbooks. Deliverable: production MLOps stack demo with CI for model training→registry→serving→monitoring and rollback.

📦 Capstone Projects (portfolio — required)

Build 3 projects (one per anchor) plus one integrated production demo:

1. **MVP Clinic Assistant (Phase 6+7+11A)** — FHIR ingestion → LLM summarization → clinician feedback; deployed, instrumented, with retention metrics.
2. **Stateful Agent for Medical QA (Phase 5+8+11B)** — RAG + HyDE + reranker + orchestrator; cost & latency optimizations.
3. **MLOps Reliability Demo (Phase 9+11C)** — full pipeline: training → registry → canary → monitoring → rollback.
4. **Production Playground** — Telegram bot + dashboard + billing + PostHog metrics + compliance artifacts (this is your showpiece).

Each project must have: repo, README, deployment, tests, monitoring, and one short case study (1-pager) showing business impact (metric improvement).
